STŘEDNÍ PRŮMYSLOVÁ ŠKOLA BRNO, PURKYŇOVA,
PŘÍSPĚVKOVÁ ORGANIZACE

TASKIFY

JAROSLAV DVOŘÁČEK

V4A

PROFILOVÁ ČÁST MATURITNÍ ZKOUŠKY
MATURITNÍ PRÁCE

BRNO 2026

Prohlášení

Prohlašuji, že:

- že jsem maturitní práci na téma ***Taskify*** vypracoval samostatně a použil jen zdroje uvedené v seznamu literatury.

Beru na vědomí, že:

- zpráva o řešení maturitní práce a základní dokumentace k aplikaci bude uložena v elektronické podobě v IS SharePoint Střední průmyslové školy Brno, Purkyňova;
- bude má maturitní práce včetně zdrojových kódů uložena v knihovně SPŠ Brno, Purkyňova, dostupná k prezenčnímu nahlédnutí;
- SPŠ Brno, Purkyňova má právo celou moji práci použít k výukovým účelům a po mém souhlasu moji práci nevýdělečně užít ke své vnitřní potřebě;
- pokud je součástí mojí práce jakýkoliv softwarový produkt, považují se za součást práce i zdrojové kódy, které jsou předmětem maturitní práce, případně soubory, ze kterých se práce skládá. Součástí práce není software/aplikace, který je pro tvorbu a následnou obhajobu práce pouze využíván za přesně definovaných podmínek.

Jméno, příjmení: Jaroslav Dvořáček

Adresa: Nádražní 299, 667 01 Židlochovice

Dne: 20.4.2026

Podpis:

Poděkování (není povinné)

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Integer rutrum, orci vestibulum ullamcorper ultricies, lacus quam ultricies odio, vitae placerat pede sem sit amet enim. Etiam posuere lacus quis dolor. Phasellus faucibus molestie nisl. Phasellus et lorem id felis nonummy placerat. Nullam sit amet magna in magna gravida vehicula.

Anotace

Cílem této maturitní práce je návrh a implementace webové aplikace Taskify, která slouží ke správě a plnění drobných úkolů v rámci lokální komunity. Aplikace propojuje uživatele, kteří potřebují s něčím pomoci, s těmi, kteří jsou ochotni úkoly plnit. Systém je postaven na platformě ASP.NET Core a využívá relační databázi spravovanou přes Entity Framework Core. Klíčovým prvkem uživatelského rozhraní je mapové zobrazení dostupných úkolů pomocí knihovny Leaflet.js, což usnadňuje prostorovou orientaci. Pro zvýšení motivace uživatelů je navržen systém gamifikace, zahrnující bodové ohodnocení, odznaky a žebříčky aktivit. Důraz je kladen také na bezpečnost uživatelských dat, která je posílena implementací volitelného dvoufázového ověření (2FA) pomocí standardu TOTP. Práce obsahuje teoretický rozbor existujících řešení, návrh architektury, popis samotné implementace a nasazení aplikace. Výsledkem je plně funkční prototyp komunitní platformy, který demonstruje využití moderních webových technologií v praxi.

Klíčová slova: webová aplikace, ASP.NET Core, Entity Framework Core, komunitní výpomoc, gamifikace, mapové zobrazení, Taskify, C#

The goal of this school-leaving thesis is the design and implementation of a web application called Taskify, which serves for managing and completing small tasks within a local community. The application connects users who need help with those who are willing to complete the tasks. The system is built on the ASP.NET Core platform and utilizes a relational database managed through Entity Framework Core. A key element of the user interface is the map display of available tasks using the Leaflet.js library, which facilitates spatial orientation. To increase user motivation, a gamification system is designed, including point scoring, badges, and activity leaderboards. Emphasis is also placed on user data security, which is enhanced by the implementation of optional two-factor authentication (2FA) using the TOTP standard. The thesis contains a theoretical analysis of existing solutions, architecture design, a description of the implementation itself, and the deployment of the application. The result is a fully functional prototype of a community platform that demonstrates the use of modern web technologies in practice.

Keywords: web application, ASP.NET Core, Entity Framework Core, community help, gamification, map display, Taskify, C#

Obsah

Úvod 9
1 Teoretická část (rešerše) 9
1.1 Cíle a metodika rešerše 9
1.2 Analýza platformy TaskRabbit 10
1.3 Analýza komunitní sítě NextDoor 10
1.4 Porovnání s platformou Taskify a identifikace inovací 10
1.4.1 Prostorová orientace prostřednictvím mapového rozhraní 11
1.4.2 Integrace gamifikačních mechanismů 11
1.4.3 Životní cyklus úkolu a správa rolí 13
2 Rozbor problému a návrh řešení 13
2.1 Výběr a zdůvodnění technologií 13
2.2 Softwarová architektura a návrhový vzor 14
2.3 Uživatelské role a stavový automat 14
2.4 Datový model a návrh databáze 16
3 Implementace Backendové vrstvy 17
3.1 Infrastruktura identity a autorizační mechanismy 17
3.2 Architektura Dependency Injection a Middleware Pipeline 18
3.3 Datový model a integrita v Entity Framework Core 19
3.4 Business logika, notifikační engine a validace 21
3.5 Životní cyklus úkolu a „Just-In-Time“ logika 22
3.6 Gamifikace a výkonová optimalizace 22
3.7 Administrace a seeding dat 26
3.8 Analýza a řešení kritických logických defektů 26
3.8.1 Popis první anomálie 26
3.8.2 Implementace nápravného opatření první anomálie 27
4 Implementace klientské vrstvy a uživatelského rozhraní 27
4.1 Architektura prezentační vrstvy a modernizace stylování 27
4.2 Integrace mapového rozhraní Leaflet.js 30
4.3 Server-Side Data Masking a ochrana lokací pro hosty 31
4.4 Analytika a vizualizace dat prostřednictvím Chart.js 33
4.5 Vizuální prezentace gamifikačních mechanismů v UI 35

4.6 Pokročilá logika interaktivních formulářů ... 36
4.6.1 Správa multimédií a klientská validace ... 36
4.6.2 Integrace externích UI komponent ... 37
4.7 Geokódování a interakce s Nominatim API ... 37
4.8 Asynchronní systém hlášení a správa incidentů ... 37
4.9 Modularita a design systému „Task Card“ ... 38
4.10 Dynamické filtrování a vyhledávací bar ... 39
4.11 Real-time správa oznámení a práce s DOMem ... 40
Závěr ... 40
Seznam ilustrací ... 41
Seznam tabulek ... 42
Seznam použitých zdrojů a literatury ... 43

Seznam použitých zkratek

- API – Application Programming Interface – Rozhraní pro programování aplikací
- CSRF – Cross-Site Request Forgery – Typ útoku zneužívající identitu uživatele
- CSS – Cascading Style Sheets – Kaskádové styly
- DI – Dependency Injection – Návrhový vzor pro vkládání závislostí
- DOM – Document Object Model – Objektový model HTML dokumentu
- DTO – Data Transfer Object – Objekt pro přenos dat mezi vrstvami aplikace
- EF Core – Entity Framework Core – Nástroj pro objektově-relační mapování
- ERD – Entity-Relationship Diagram – Model entit a vztahů
- GDPR – General Data Protection Regulation – Nařízení o ochraně osobních údajů
- GIF – Graphics Interchange Format – Grafický formát pro rastrovou grafiku
- GPS – Global Positioning System – Globální polohový systém
- GUID – Globally Unique Identifier – Globálně unikátní identifikátor
- HTML – HyperText Markup Language – Hypertextový značkovací jazyk
- IoC – Inversion of Control – Architektonický princip inverze řízení
- LINQ – Language Integrated Query – Nástroj pro dotazování nad daty v .NET
- MVC – Model-View-Controller – Architektonický vzor návrhu
- ORM – Object-Relational Mapping – Objektově-relační mapování
- PBKDF2 – Password-Based Key Derivation Function 2 – Standard pro hashování hesel
- RFC – Request for Comments – Technické standardy internetu
- SMTP – Simple Mail Transfer Protocol – Protokol pro odesílání e-mailů
- SQL – Structured Query Language – Strukturovaný dotazovací jazyk
- TOTP – Time-based One-Time Password – Časově omezené jednorázové heslo

- UI – User Interface – Uživatelské rozhraní
- UML – Unified Modeling Language – Unifikovaný modelovací jazyk
- UX – User Experience – Uživatelský prožitek
- XSS – Cross-Site Scripting – Typ zranitelnosti webových aplikací
- JSON – JavaScript Object Notation – Lehký datový formát pro výměnu dat
- JS – JavaScript
- HTTP – HyperText Transfer Protocol
- URL – Uniform Resource Locator
- XP – Experience Points – Zkušenostní body

Slovník pojmů

- ASP.NET Core – Moderní, vysoce výkonný a multiplatformní framework od společnosti Microsoft určený pro vývoj webových aplikací a služeb.
- C# – Moderní, silně typovaný a objektově orientovaný programovací jazyk, který je hlavním jazykem platformy .NET.
- Clean Code – Metodika a soubor pravidel pro psaní zdrojového kódu tak, aby byl snadno čitelný, udržovatelný a srozumitelný pro ostatní vývojáře.
- Enum – Výčtový typ v programování, který umožňuje proměnné nabývat pouze jedné z pevně definovaných celočíselných hodnot (konstant).
- Razor Pages – Programovací model v rámci ASP.NET Core zaměřený na stránky, který propojuje HTML kód s logikou v C#.
- SQLite – Relační databázový systém, který nefunguje jako server, ale ukládá veškerá data do jednoho běžného souboru na disku.
- Stack – Termín označující soubor všech technologií (programovací jazyky, databáze, knihovny), které byly zvoleny pro vytvoření aplikace.
- State Machine – Stavový automat – logický model, který řídí přechody mezi různými stavy objektu (např. životní cyklus úkolu) na základě definovaných pravidel.
- Claims – Datové atributy identity uživatele (např. role) uložené v autentizačním kontextu.
- Denormalizace – Strategie úpravy databáze pro zvýšení rychlosti čtení statistik.
- Environment Variables – proměnné prostředí serveru sloužící k bezpečnému uložení hesel a klíčů.
- Just-In-Time Notifikace – Upozornění generované v momentě, kdy uživatel vstoupí na relevantní stránku.
- Loose Coupling – Volná vazba mezi komponentami systému pro snazší údržbu a testování.
- Middleware – Softwarová komponenta zpracovávající požadavky v rámci pipeline serveru.
- TempData – Dočasné úložiště v ASP.NET Core pro předávání zpráv mezi přesměrováními.

- Bar chart – Sloupcový graf určený pro porovnávání hodnot v jednotlivých kategoriích či časových úsecích.
- Call to Action (CTA) – Vizuálně výrazný prvek rozhraní (např. tlačítko), který má za cíl přimět uživatele k provedení určité akce (např. registrace).
- Doughnut chart – Typ koláčového grafu s prázdným středem (tvar prstence), často využívaný pro moderní zobrazení proporcí.
- Flexbox – Technologie v CSS pro efektivní uspořádání a zarovnání prvků na webové stránce nezávisle na velikosti displeje.
- Grid systém – Rozvržení rozhraní založené na síti sloupců a řádků, sloužící k responzivní adaptaci obsahu.
- Konverzní trychtýř – Promyšlená cesta v aplikaci, která provádí uživatele kroky nutnými k dosažení stanoveného cíle (např. k vytvoření účtu).
- Line chart – Spojnicový graf využívaný k zobrazení změn a trendů v kontinuálním čase.
- Login Gate – Prvek UI (modální okno či overlay), který blokuje skrytý obsah a podmiňuje jeho zobrazení autentizací uživatele.
- Marker – Grafická značka reprezentující konkrétní bod (např. úkol) vykreslená na interaktivní mapě.
- Obfuskace – Technika úpravy dat (např. úmyslný posun souřadnic), která snižuje jejich přesnost a brání zneužití citlivých informací.
- Popup – Vyskakovací okno s dodatečnými informacemi, které se aktivuje po interakci uživatele s prvkem na stránce (např. kliknutím na marker v mapě).
- Progress bar – Vizuální indikátor stavu či pokroku, typicky používaný pro zobrazení míry splnění či získaných zkušenostních bodů (XP).
- Rarity – Atribut vzácnosti určující exkluzivitu odměny (achievementu), který se na frontendu projevuje specifickým stylováním.
- Responzivita – Vlastnost aplikace, která zajišťuje automatické přizpůsobení designu a zobrazení prvků parametrům koncového zařízení (např. mobil, PC).
- Server-side data masking – Bezpečnostní přístup, při němž se citlivá data (např. exaktní adresa) maskují či znehodnocují přímo na serveru před jejich odesláním klientovi.
- Šablona (Template) – Předpřipravený formát struktury kódu (často s "placeholdery" pro data), který slouží k dynamickému generování finálního obsahu.
- Base64 – Způsob kódování dat (např. obrázků) do textového řetězce, který lze vložit přímo do HTML.
- Fetch API – Moderní rozhraní v JavaScriptu pro odesílání síťových požadavků na server.
- Geokódování – Proces převodu textové adresy na zeměpisné souřadnice (šířku a délku).
- Nominatim – Vyhledávací nástroj (engine) pro data z OpenStreetMap, sloužící ke geokódování.
- Partial View – Částečný pohled v ASP.NET Core, který obsahuje pouze kus HTML kódu pro vložení do jiné stránky.

- Reverzní geokódování – Proces převodu zeměpisných souřadnic na srozumitelnou textovou adresu.
- CSRF Token – Bezpečnostní klíč sloužící k ověření, že požadavek na server odeslal skutečný uživatel z dané aplikace.
- Achievement (Odznak) – Vizuální ocenění uživatele za splnění specifických podmínek.
- Code-First – Přístup k vývoji databáze, kdy je schéma generováno automaticky na základě C# tříd.
- Data Annotations – Atributy v C# (např. [Required]) sloužící k validaci datového modelu.

Úvod

V současné uspěchané době se často vytrácí mezilidská solidarita a vzájemná výpomoc v rámci lokálních komunit. Lidé mnohdy potřebují pomoci s drobnými každodenními činnostmi – ať už se jedná o nákup pro seniora, venčení domácího mazlíčka, nebo drobnou opravu v domácnosti. Zároveň existuje ve společnosti řada jedinců, kteří mají volný čas a ochotu pomáhat svému okolí, avšak chybí jim efektivní nástroj, jak tyto příležitosti k pomoci vyhledat. Z tohoto důvodu bylo přistoupeno k návrhu a vývoji webové aplikace Taskify.

Hlavním cílem této maturitní práce je vytvořit dynamickou a uživatelsky přívětivou webovou platformu, která tento problém řeší. Aplikace Taskify je navržena tak, aby intuitivně propojovala poptávku po drobných úkolech s jejich nabídkou v rámci specifické geografické oblasti. Byl kladen důraz na to, aby byl celý proces od zadání úkolu až po jeho splnění co nejvíce transparentní a motivující pro obě zúčastněné strany.

Pro dosažení vytyčených cílů byly stanoveny následující dílčí kroky implementace a návrhu:

1. Analyzovat existující řešení na trhu a identifikovat jejich silné a slabé stránky ve vztahu ke komunitnímu využití.
2. Navrhnout intuitivní uživatelské rozhraní s důrazem na geografickou orientaci, čehož bude dosaženo primárně integrací interaktivních map.
3. Implementovat robustní backendovou vrstvu s využitím moderního frameworku ASP.NET Core a zajistit bezpečné ukládání a zpracování dat relačního charakteru.
4. Zakomponovat do systému prvky gamifikace, jako jsou bodování, žebříčky a odznaky, pro dlouhodobé udržení pozornosti a motivace uživatelů k aktivní participaci na platformě.

Výsledný produkt má sloužit jako plně funkční prototyp prokazující schopnost samostatně vyvinout komplexní softwarové dílo, které reaguje na reálnou společenskou potřebu a plně využívá aktuální technologické standardy v oblasti vývoje webových aplikací.

1 Teoretická část (rešerše)

1.1 Cíle a metodika rešerše

Před samotným návrhem architektury a zahájením implementačních prací byla provedena důkladná rešerše trhu a analýza existujících platforem, které se zabývají zprostředkováním práce či komunitní výpomocí. Cílem této analýzy bylo identifikovat běžně používané návrhové vzory, odhalit nedostatky současných řešení v kontextu lokální sousedské výpomoci a precizně definovat prostor pro inovace, které aplikace Taskify přináší.

1.2 Analýza platformy TaskRabbit

Jedním z nejznámějších globálních zástupců v oblasti zprostředkování drobných úkolů a prací je platforma TaskRabbit. Tento systém je primárně orientován na komerční poskytování služeb. Uživatelé zde poptávají profesionální či poloprofesionální pomoc (například montáž nábytku, hloubkový úklid nebo stěhování) a takzvaní "Taskeři" (vykonavatelé úkolů) jsou za svou práci finančně ohodnoceni. Platforma si z každé realizované transakce účtuje provizi, na čemž je postaven její obchodní model.

Silnou stránkou tohoto řešení je vysoká spolehlivost, propracovaný systém hodnocení a detailní verifikace uživatelů, čímž platforma buduje silnou vzájemnou důvěru. Aplikace disponuje robustním uživatelským rozhraním a širokou škálou kategorizovaných služeb.

Zásadním nedostatkem pro lokální komunity je však právě její striktní komercializace. Platforma není koncepčně navržena pro nezištnou sousedskou výpomoc. Proces registrace pro vykonavatele je administrativně složitý a primární orientace na finanční zisk zcela potlačuje sociální a komunitní aspekt pomoci. Ve webovém rozhraní rovněž absentují výraznější prvky gamifikace, které by uživatele motivovaly k plnění drobných, bezúplatných laskavostí.

1.3 Analýza komunitní sítě NextDoor

Dalším analyzovaným segmentem softwaru jsou platformy zaměřené primárně na komunitu, reprezentované například sociální sítí Nextdoor. Jedná se o platformu pro obyvatele konkrétních čtvrtí a sousedství, která umožňuje sdílení lokálních informací, organizaci událostí a v omezené míře také poptávku či nabídku sousedské pomoci.

Platforma těží ze silného důrazu na budování komunity a regionálního zaměření. Uživatelé komunikují výhradně s lidmi ve svém bezprostředním okolí, což výrazně zvyšuje relevanci zobrazovaného obsahu.

Ve vztahu ke správě úkolů je však Nextdoor nedostačující. Aplikace funguje primárně jako nástěnka a diskusní fórum. Modul pro zadávání úkolů netvoří jádro systému a postrádá strukturovaný životní cyklus požadavku (od vytvoření, přes přijetí, až po potvrzení o splnění). Poptávky po pomoci se často ztrácejí v záplavě nesouvisejících příspěvků a zásadně zde chybí přehledné mapové zobrazení konkrétních žádostí.

1.4 Porovnání s platformou Taskify a identifikace inovací

Na základě zjištění z výše provedené analýzy byl koncept webové aplikace Taskify architektonicky navržen tak, aby sloučil výhody strukturovaných zadávacích systémů se

sociálním a primárně nekomerčním rozměrem komunitních sítí. Byla identifikována a následně implementována funkční specifika, kterými se Taskify výrazně odlišuje od zkoumaných alternativ. Dalším inovativním prvkem oproti analyzovaným platformám je integrace pokročilého zabezpečení účtů. Taskify umožňuje uživatelům aktivaci dvoufázového ověření prostřednictvím externích autentizačních aplikací, což minimalizuje riziko zneužití profilu i v případě úniku hesla.

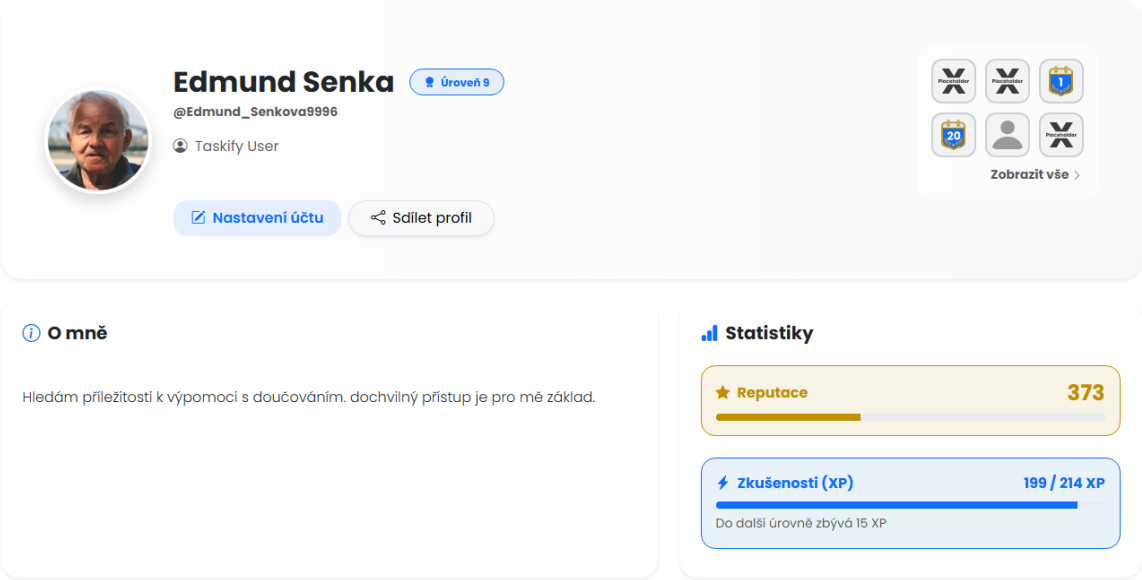
1.4.1 Prostorová orientace prostřednictvím mapového rozhraní

Zatímco absolutní většina analyzovaných platforem zobrazuje úkoly primárně ve formě textových lineárních seznamů, Taskify integruje jako svůj ústřední navigační prvek interaktivní mapu, k jejímuž vykreslování byla zvolena JavaScriptová knihovna Leaflet.js. Toto technologické rozhodnutí vychází z předpokladu, že pro efektivní sousedskou výpomoc je geografická blízkost naprosto rozhodujícím faktorem. Uživatel po přihlášení do aplikace okamžitě vidí prostorové rozložení úkolů ve svém okolí. Vizuální reprezentace dat na mapě výrazně snižuje psychologickou bariéru pro zapojení a usnadňuje orientaci.

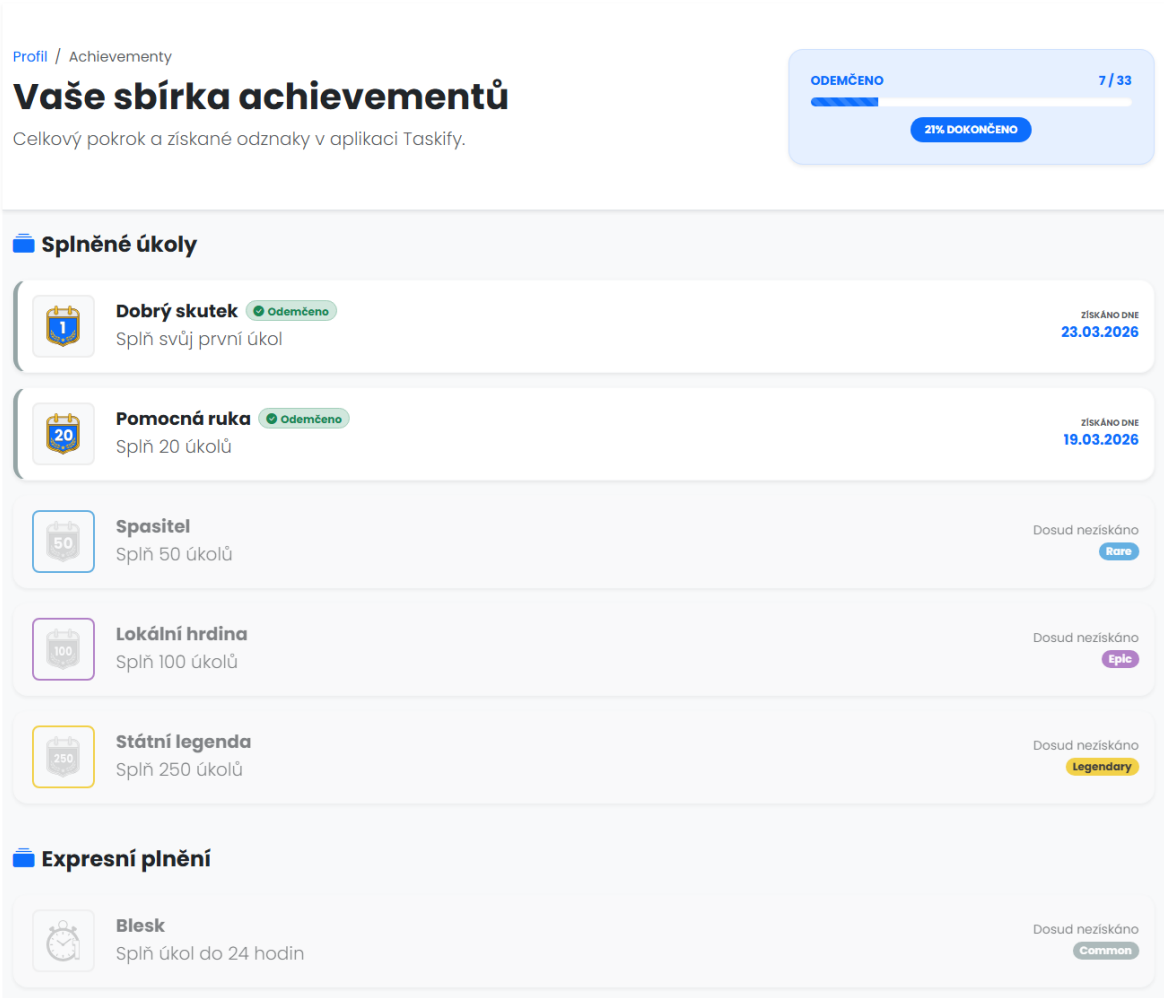
1.4.2 Integrace gamifikačních mechanismů

Vzhledem k tomu, že v aplikaci Taskify chybí přímá finanční odměna, bylo nutné najít jiný způsob, jak uživatele motivovat k pomoci ostatním. Do návrhu byly proto zahrnuty herní prvky (tzv. gamifikace). Hlavním cílem je vytvořit zajímavé prostředí, které bude uživatele bavit a motivovat k dlouhodobé aktivitě.

Mezi tyto prvky patří komplexní bodovací systém, veřejné žebříčky a odemykání vizuálních odznaků (tzv. achievementů). Tyto odznaky nejsou vázány pouze na jednu kategorii úkolů, ale odráží celkovou aktivitu uživatele. Jsou přidělovány za dosažení důležitých milníků, jako je například splnění určitého počtu úkolů. Pro zvýšení atraktivity byl navržen i systém skrytých odznaků, které jsou získávány za specifické akce – například za aktivitu v brzkých ranních hodinách nebo za nahrání profilové fotografie uživatele. Tento přístup má za cíl vzbudit v uživatelích zvědavost a podpořit jejich chuť objevovat všechny funkce aplikace. Vizuální reprezentace celkového pokroku, úrovně a reputace je integrována přímo do uživatelského rozhraní (viz Obr. 1.4-1: Uživatelský profil). Pro detailní sledování herních milníků a odemčených odznaků pak uživateli slouží samostatná galerie (viz Obr. 1.4-2: Sbírka achievementů).



Obr. 1.4-1: Uživatelský profil



Obr. 1.4-2: Sbírka achievementů

1.4.3 Životní cyklus úkolu a správa rolí

Na rozdíl od běžných diskusních skupin na sociálních sítích je v aplikaci Taskify kladen velký důraz na jasný přehled o tom, v jakém stavu se daný úkol nachází. Každý úkol vložený do systému má svůj přesně daný postup. V databázi je sledováno, zda je úkol nový, zda jej již někdo přijal, nebo zda je již hotový a potvrzený. Tento systém zabraňuje tomu, aby se na jeden úkol vydalo více lidí najednou, a celkově zvyšuje přehlednost platformy.

Bezpečnost v rámci komunity je zajištěna rozdělením uživatelů do tří rolí: Administrátor, Uživatel a Host. Administrátor má k dispozici nástroje pro kontrolu obsahu a může blokovat uživatele, kteří by porušovali pravidla. Toto bylo vyhodnoceno jako klíčový požadavek pro to, aby aplikace zůstala bezpečným místem pro vzájemnou sousedskou výpomoc.

2 Rozbor problému a návrh řešení

V této kapitole jsou detailně analyzovány teoretické i praktické aspekty návrhu aplikace Taskify. Pozornost je věnována výběru technologického stacku, dekompozici systému na jednotlivé role a stavy a preciznímu návrhu databázové struktury s ohledem na integritu a výkon. Celý návrh směřuje k vytvoření robustní a škálovatelné platformy, která bude snadno udržitelná a uživatelsky přívětivá.

2.1 Výběr a zdůvodnění technologií

Pro realizaci projektu byl zvolen ekosystém společnosti Microsoft, konkrétně platforma .NET 9. Toto rozhodnutí bylo podloženo potřebou vysokého výkonu, silné typové kontroly a robustního zabezpečení, které je pro komunitní aplikaci klíčové. Jako hlavní vývojový framework byl vybrán ASP.NET Core, který oproti starším verzím nabízí plnou multiplatformnost a optimalizovaný kód, což se projevuje především v rychlé odezvě serverové části aplikace. V rámci frameworku ASP.NET Core Identity bylo plně využito vestavěných mechanismů pro podporu vícefázové autentizace a bezpečného generování obnovovacích kódů. Nedílnou součástí zvoleného řešení je Entity Framework Core (EF Core), který slouží jako vrstva objektově-relačního mapování (ORM). EF Core byl do projektu integrován především pro svou schopnost automaticky chránit aplikaci před útoky typu SQL Injection a pro efektivní metodiku Code-First, která dovoluje verzovat změny v databázi obdobně jako zdrojový kód.

Jako databázové úložiště byla zvolena relační databáze SQLite. Ačkoliv se jedná o souborově orientovanou databázi, pro projekt Taskify nabízí specifické výhody, které v daném měřítku převyšují nasazení robustních SQL serverů. Hlavním přínosem je nulová potřeba konfigurace na straně hostitelského stroje a vysoká portabilita – veškerá data jsou uložena v jediném souboru přímo v adresáři projektu. Toto řešení umožňuje okamžité spuštění aplikace na jiném zařízení bez nutnosti instalovat složité databázové služby, což je pro účely odevzdání maturitní

práce ideální volba. Zároveň byla zachována plná kompatibilita s EF Core, což v budoucnu umožňuje snadný přechod na výkonnější serverové řešení pouhou změnou konfiguračního řetězce.

Klientská část aplikace využívá moderní frontendové knihovny, které jsou průmyslovým standardem. Pro mapové rozhraní byla vybrána knihovna Leaflet.js, která vyniká svou lehkostí a vysokým výkonem při vykreslování velkého množství značek (markerů). Vizuální stránka a responzivita jsou zajištěny frameworkem Bootstrap 5, který využívá moderní CSS vlastnosti pro bezproblémové zobrazení aplikace například i na mobilních zařízeních. Doplňkově byla integrována knihovna Chart.js, která slouží k vizualizaci statistik a aktivit uživatele, čímž výrazně přispívá k přehlednosti implementovaných gamifikačních prvků.

2.2 Softwarová architektura a návrhový vzor

V projektu je uplatněn architektonický vzor Razor Pages, který představuje moderní součást frameworku ASP.NET Core. Tento přístup byl vybrán jako efektivní alternativa k tradičnímu vzoru MVC (Model-View-Controller). Zatímco v MVC je logika často rozmělněna mezi mnoho kontrolerů, Razor Pages izolují logiku a zobrazení pro každou konkrétní stránku do samostatné funkční jednotky. Tento koncept implementuje návrhový vzor Page Controller, který vede k čistšímu zdrojovému kódu a snazší údržbě celého systému.

Každá stránka v aplikaci je tvořena souborem pro definici zobrazení (HTML/Razor) a přidruženou třídou v C#, která obsluhuje vstupní data a komunikaci s databází. Toto úzké propojení eliminuje potřebu složitého směrování a zjednodušuje mapování mezi požadavkem uživatele a reakcí serveru. Výsledkem je vysoce modulární architektura, kde jsou jednotlivé funkce aplikace, jako je správa úkolů nebo profil uživatele, striktně odděleny, což zamezuje vzniku nepřehledných a těžko testovatelných částí kódu.

2.3 Uživatelské role a stavový automat

V návaznosti na analýzu trhu a identifikované potřeby lokálních komunit (viz kapitola 1.4) byl vypracován formální model uživatelských oprávnění a životního cyklu požadavku. Tento model přetavuje teoretické požadavky na bezpečnost a přehlednost do konkrétních technických struktur, které tvoří logické jádro aplikace. Hierarchie uživatelů je rozdělena do tří rolí, přičemž každá disponuje specifickou sadou akcí definovanou v rámci aplikační logiky.

V rámci návrhu je dále rozlišováno mezi logickými rolemi Zadavatele a Dobrovolníka, ačkoliv v implementaci jsou obě tyto role zastoupeny jedním fyzickým uživatelským účtem. Toto rozdělení slouží k precizní definici oprávnění v rámci životního cyklu úkolu a je klíčové pro správné zobrazení uživatelských interakcí v systému.

Obr. 2.3-1: UML Use Case diagram

Zatímco role Administrátor a Uživatel vyžadují plnou autentizaci prostřednictvím systému Identity, role Hosta je implementována jako anonymní přístup s právem „read-only“ k mapovým podkladům. Pro definici stavů úkolů a úrovní priority byl v návrhu důsledně uplatněn datový typ Enum. Z hlediska principů Clean Code se jedná o klíčové rozhodnutí, které přináší typovou bezpečnost a předchází logickým chybám v kódu. Na úrovni databáze SQLite jsou tyto hodnoty ukládány jako celá čísla, což radikálně urychluje vyhledávání a filtrování dat oproti porovnávání textových řetězců. Logika aplikace je následně řízena striktním stavovým automatem (TaskStatus), který definuje přechody mezi stavy Open, InProgress, Completed a Archived. Tento automat zaručuje, že úkol nemůže být například znovu přijat jiným uživatelem, pokud se již nachází ve stavu řešení, a že potvrzení o splnění může provést pouze aktuálně přiřazený řešitel.

2.4 Datový model a návrh databáze

Návrh databáze byl realizován s důrazem na referenční integritu a zamezení redundance dat. Celé schéma je postaveno na několika propojených entitách, přičemž ústřední roli hraje entita TaskItem. Tento název byl zvolen záměrně, aby se předešlo kolizím se systémovou třídou Task v knihovnách .NET, což zvyšuje čitelnost zdrojového kódu a usnadňuje orientaci vývojáře v projektu. Jako primární klíče byly pro všechny aplikační entity zvoleny celočíselné identifikátory s automatickou inkrementací, které v databázi SQLite vykazují vyšší výkon při indexaci a řazení oproti identifikátorům typu GUID.

Vztahy mezi entitami jsou v databázovém kontextu ošetřeny pomocí pravidel explicitních cizích klíčů a definovaných vazeb 1:N. Například entita uživatele je s úkoly propojena dvěma nezávislými cestami – jako autor (CreatedTasks) a jako řešitel (AssignedTasks). Pro zachování historické integrity dat byla nastavena pravidla pro mazání záznamů tak, aby při odstranění účtu uživatele nedošlo ke smazání jeho vytvořených úkolů, ale pouze k uvolnění těch, které měl právě rozpracované (DeleteBehavior.SetNull). Tento mechanismus zajišťuje, že komunitní aktivita zůstane v systému zachována i při fluktuaci uživatelské základny.

Součástí datového návrhu je i systém gamifikace a uživatelských statistik, který je integrován přímo do modelu uživatele. Sledovány jsou parametry jako reputace, bodové ohodnocení a dosažená úroveň, které jsou dynamicky přepočítávány na základě aktivity v systému. Tento model je dále rozšířen o komplexní správu notifikačních preferencí, což uživatelům umožňuje detailně konfigurovat, které události mají spouštět e-mailová upozornění. Tímto způsobem je dosaženo vysoké personalizace a uživatelského komfortu při používání aplikace, přičemž veškerá data jsou ukládána s ohledem na budoucí rozšiřitelnost systému o nové typy achievementů a sociálních interakcí.

3 Implementace Backendové vrstvy

V této kapitole je proveden hloubkový technický rozbor implementace serverové části aplikace. Pozornost je věnována konkrétním architektonickým rozhodnutím v rámci frameworku .NET 9, integraci bezpečnostních protokolů a optimalizaci datových toků, které zajišťují stabilitu a škálovatelnost celého systému. Celé řešení je postaveno na moderní architektuře Razor Pages, která umožňuje efektivní separaci business logiky od prezentační vrstvy při zachování vysoké modulárnosti systému.

3.1 Infrastruktura identity a autorizační mechanismy

Základem bezpečnosti aplikace je integrovaný systém ASP.NET Core Identity, který byl do projektu přidán pomocí techniky scaffoldingu a následně hloubkově upraven pro potřeby komunitní sítě. Autentizace je postavena na stavových souborech cookies, které zajišťují perzistenci uživatelské relace. Po úspěšném ověření uživatele pomocí algoritmu PBKDF2 vygeneruje server šifrovanou a kryptograficky podepsaná cookie. Middleware Identity zajišťuje, že její obsah je pro klienta nečitelný a jakýkoliv pokus o modifikaci vede k její okamžité neplatnosti. Z hlediska bezpečnosti je klíčové využití příznaku *HttpOnly*, který znemožňuje přístup k údajům pomocí klientských skriptů, čímž je aplikace chráněna proti útokům typu XSS. Dále je uplatňována politika *SameSite*, která tvoří efektivní obranu proti CSRF útokům.

Klíčovým konceptem jsou zde tzv. **Claims** (tvrzení). Jedná se o drobné datové jednotky nesoucí informace o uživateli (ID, jméno, role), které jsou uloženy přímo v autentizačním kontextu. To radikálně snižuje zátěž na databázi, protože server má identitu uživatele okamžitě k dispozici v operační paměti. Pro uživatele vyžadující vyšší standard zabezpečení byla integrována podpora TOTP (Time-based One-Time Password) dle standardu RFC 6238. Backend generuje unikátní 160bitový tajný klíč, který je uživateli prezentován ve formě QR kódu. Tento klíč je v databázi uložen v šifrované podobě pomocí vestavěného poskytovatele ochrany dat (Data Protection API). Algoritmus pracuje na principu hašování aktuálního času a tajného klíče, což generuje šestimístný kód platný po dobu 30 sekund. Implementace v Taskify navíc zahrnuje ochranu proti "replay" útokům, kdy systém nepovolí použití stejného kódu dvakrát v rámci jednoho časového okna.

```
public override IdentityError PasswordTooShort(int length)
{
    return new IdentityError
    {
        Code = nameof(PasswordTooShort), Description = $"Heslo
musí mít alespoň {length} znaků."
    };
}
```

Ukázka kódu 3.1-1: Přepsání metody PasswordTooShort

Standardní Identity systém generuje chybová hlášení v angličtině, což bylo pro českou komunitní aplikaci vyhodnoceno jako kritická bariéra. Byla proto implementována třída *CzechIdentityErrorDescriber*, která dědí ze základní třídy IdentityErrorDescriber. Pomocí metody přepisování (override) byly všechny virtuální metody upraveny tak, aby vracely české textové řetězce. Tím bylo dosaženo profesionálního rozhraní, které uživatele registrací provází srozumitelně a zvyšuje důvěryhodnost aplikace.

3.2 Architektura Dependency Injection a Middleware Pipeline

Aplikace důsledně využívá vzor Dependency Injection (DI) pro zajištění udržovatelnosti kódu. V souboru *Program.cs* je implementován IoC kontejner, který spravuje životní cyklus komponent. Byly využity různé typy registrací:

- **Transient (AddTransient):** Služba EmailSender je registrována jako Transient. To znamená, že pro každé odeslání e-mailu je vytvořena nová, čistá instance, která je po dokončení práce okamžitě uvolněna z paměti. Toto řešení je ideální pro lehké, bezestavové služby.
- **Scoped (AddScoped):** Klíčové služby jako NotificationService, AchievementService, LevelingService a UserService jsou registrovány jako Scoped. Instance je vytvořena jednou pro každý HTTP požadavek a je sdílena napříč všemi komponentami v rámci tohoto požadavku (včetně databázového kontextu ApplicationDbContext). To zajišťuje, že všechny operace v rámci jednoho kliknutí uživatele pracují se stejnou sadou dat a stejnou transakcí.
- **Singleton:** Používá se pro služby, které mají existovat po celou dobu běhu aplikace (např. systémové logování či globální konfigurace).

Scoped služby jsou sdíleny v rámci jednoho HTTP požadavku, což zajišťuje konzistenci dat a transakcí. Tento přístup umožňuje tzv. Loose Coupling (volnou vazbu), díky čemuž jsou jednotlivé části systému nezávislé a snadno testovatelné.

Pořadí volání middleware v rámci Pipeline je v projektu nastaveno tak, aby byla prioritizována bezpečnost:

1. **UseRouting**: První krok, který identifikuje cílový PageModel na základě URL adresy.

2. **UseAuthentication**: Klíčový bod, kde se dešifrují cookies a zjišťuje se identita uživatele.
3. **UseAuthorization**: Kontrola přístupových práv identifikovaného uživatele. Pokud se tyto dva kroky prohodí, aplikace přestane fungovat, protože autorizace by kontrolovala práva anonymního uživatele.
4. **UseRequestLocalization**: Nastavení kultury prostředí na *cs-CZ*. Toto je nezbytné pro správné parsování čísel (např. desetinná čárka vs. tečka v GPS souřadnicích).

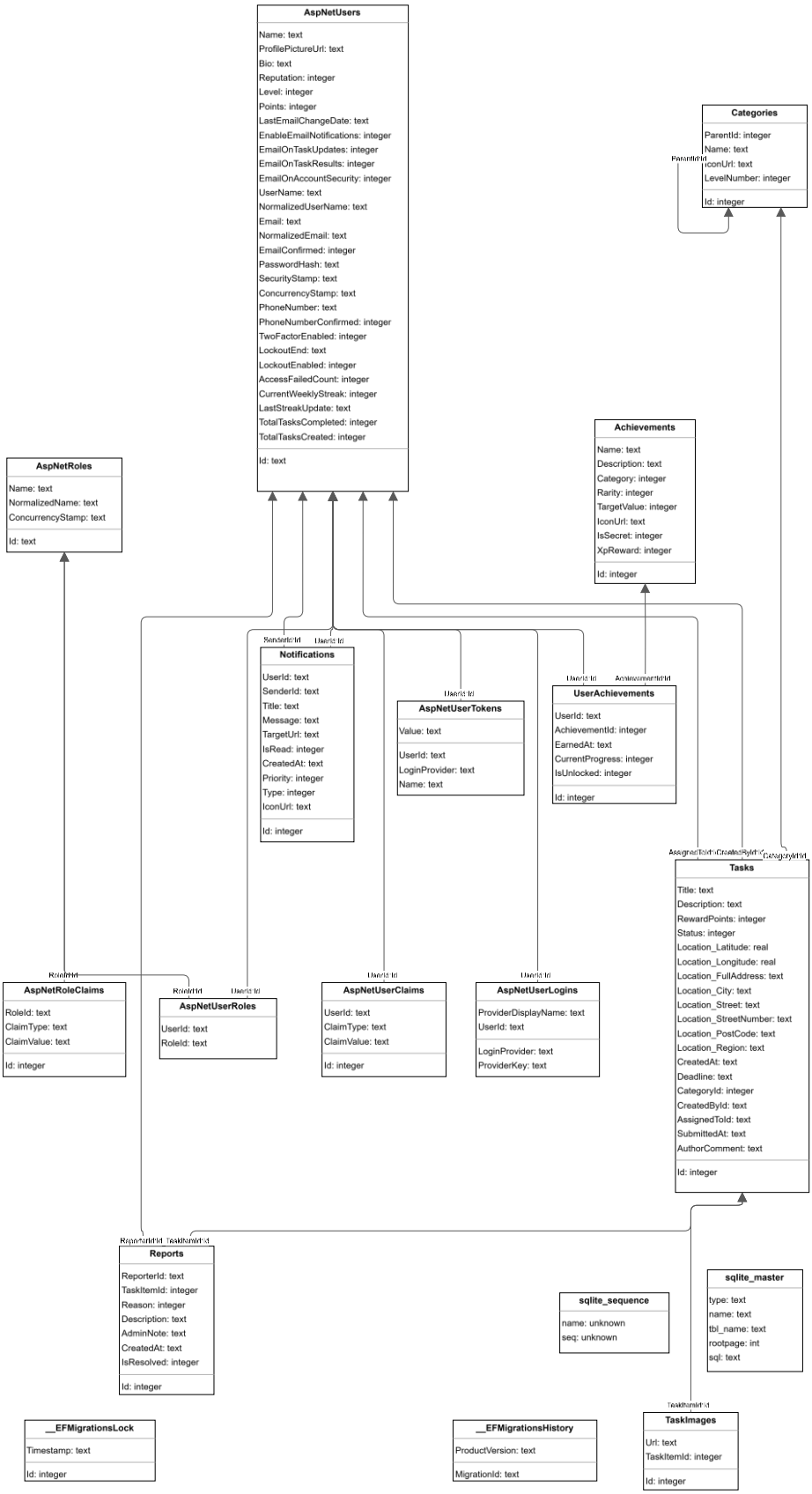
Specifickým rozhodnutím bylo umístění *UseStaticFiles* pro složku *uploads* před autorizační blok. Tím bylo docíleno toho, že obrázky k úkolům jsou dostupné na mapě i pro anonymní návštěvníky, což odpovídá otevřenému charakteru komunitní platformy.

3.3 Datový model a integrita v Entity Framework Core

Databázová vrstva využívá Entity Framework Core s metodikou Code-First. Schéma je generováno přímo z C# tříd, přičemž důraz byl kladen na referenční integritu. Aplikace implementuje pokročilý hierarchický model kategorií s třemi úrovněmi hloubky. V databázi je toto realizováno rekurzivní vazbou (Self-Referencing Relationship), kdy tabulka *Categories* obsahuje cizí klíč *ParentId* odkazující na primární klíč stejné tabulky. Tato architektura umožňuje vytvářet komplexní stromové struktury (např. Úklid → Sezónní úklid → Odklízení sněhu).

V rámci datového modelu entity TaskItem byla pro uložení lokalizačních údajů využita pokročilá technika Entity Frameworku zvaná Owned Types. Třída AddressInfo, která zapouzdřuje vlastnosti jako město, ulice a geografické souřadnice, netvoří v databázi samostatnou tabulku, ale její atributy jsou automaticky mapovány přímo do sloupců tabulky úkolů. Tímto přístupem je dosaženo vysoké logické čistoty kódu a objektově orientovaného návrhu, aniž by docházelo k degradaci výkonu databázových dotazů z důvodu nadbytečných spojování tabulek (JOIN). Z pohledu SQL schématu se tak AddressInfo jeví jako integrální součást záznamu úkolu, což zjednodušuje údržbu datové integrity.

Vizualizace kompletní logické struktury databáze, vygenerovaná přímo z produkčního schématu SQLite schématu pomocí vývojového prostředí JetBrains Rider, je znázorněna v následujícím ER diagramu (viz Obr. 3.3-1: ER Diagram databáze Taskify).



Obr. 3.3-1: ER Diagram databáze Taskify

Při dotazování backend využívá techniku Eager Loading (pomocí metody .Include()), aby minimalizoval počet dotazů do databáze. To zajišťuje bleskovou odezvu i při zobrazení složitých navigačních prvků a seznamů.

V rámci konfigurace Fluent API byla definována striktní pravidla:

- **DeleteBehavior.Restrict:** Použito pro vazbu mezi kategorií a úkolem. Databáze fyzicky nedovolí smazat kategorii, pokud v ní existují aktivní úkoly. Tím je zabráněno vzniku „sirotčích“ záznamů.
- **DeleteBehavior.SetNull:** Použito pro vazbu mezi uživatelem a jeho historií plnění. Pokud účet uživatele bude administrátorem smazán, úkoly v systému zůstanou pro statistiky komunity zachovány, ale identifikátor řešitele bude nahrazen hodnotou null. Tato logika zajišťuje historickou integritu komunitních dat.

Během vývoje byla využívána technologie migrací (*EF Core Migrations*), která umožnila plynulý vývoj schématu paralelně s programováním business logiky. Každá změna struktury byla zachycena jako migrační krok, což zajistilo, že struktura databáze SQLite přesně odpovídá požadavkům aktuální verze kódu. Všechna validační pravidla jsou definována přímo v modelech pomocí *DataAnnotations* (např. *[Required]* a *[StringLength]*), což tvoří první linii obrany proti nekonzistentním datům dříve, než se dostanou do fyzického úložiště.

3.4 Business logika, notifikační engine a validace

Business logika je oddělena do samostatných služeb, což zvyšuje přehlednost projektu. Jednou z nejnáročnějších částí byla implementace *NotificationService*, která zajišťuje distribuci upozornění v reálném čase. Pro odesílání e-mailů byl zvolen asynchronní vzor „Fire-and-Forget“ s využitím *Task.Run*. Aby nedocházelo k pádům aplikace po skončení HTTP požadavku, využívá proces na pozadí *IServiceScopeFactory* k vytvoření izolovaného kontextu služeb. *NotificationService* obsahuje navíc i algoritmus pro kontrolu duplicit. Před odesláním každé notifikace systém provede kontrolu existence identické zprávy pro daného uživatele v posledních 5 minutách. Pokud je detekována shoda, e-mail se neodesílá. Tato ochrana je kritická pro udržení dobré reputace odesílací domény a zabránění obtěžování uživatelů spamem.

```
_ = Task.Run(async () => {
    using (var scope = _scopeFactory.CreateScope())
    {
        var scopedEmailSender =
scope.ServiceProvider.GetRequiredService<IEmailSender>();
        try {
            await scopedEmailSender.SendEmailAsync(user.Email,
$"Taskify: {title}", emailBody);
```

Ukázka kódu 3.4-1: Kód s Task.Run a vytvořením nového _scopeFactory.CreateScope()

Samotné odesílání zpráv zajišťuje *EmailSender* přes protokol SMTP na portu 587. Z hlediska bezpečnosti jsou citlivé údaje (e-mail a heslo odesílatele) v produkčním prostředí Coolify načítány z proměnných prostředí (Environment Variables), nikoliv ze statických souborů. Vizuální stránka e-mailů je definována v *EmailTemplates.cs*, kde jsou dynamicky generována HTML těla zpráv. Integrita dat je dále vynucována pomocí regulárních výrazů v *AppConstants.cs*, které validují formáty e-mailů a českých telefonních čísel. Pro komunikaci mezi přesměrováními na frontendu je využit mechanismus *TempData*, který uchovává stavové zprávy v šifrované cookie do jejich prvního přečtení.

3.5 Životní cyklus úkolu a „Just-In-Time“ logika

Správa úkolů je v systému Taskify implementována jako stavový stroj řízený typem *TaskStatus*. Přechody mezi stavy (např. přijetí úkolu či jeho odevzdání) jsou realizovány jako atomické operace, což v prostředí EF Core zaručuje, že nedojde k nekonzistentním stavům při souběhu požadavků více uživatelů. Unikátním prvkem backendu je správa vypršených termínů pomocí **Just-In-Time** kontroly. Namísto náročných serverových časovačů provádí systém kontrolu v momentě, kdy uživatel navštíví svůj profil. Pokud jsou detekovány úkoly po deadlinu, backend okamžitě vygeneruje notifikace a nabídne uživateli prodloužení termínu nebo archivaci.

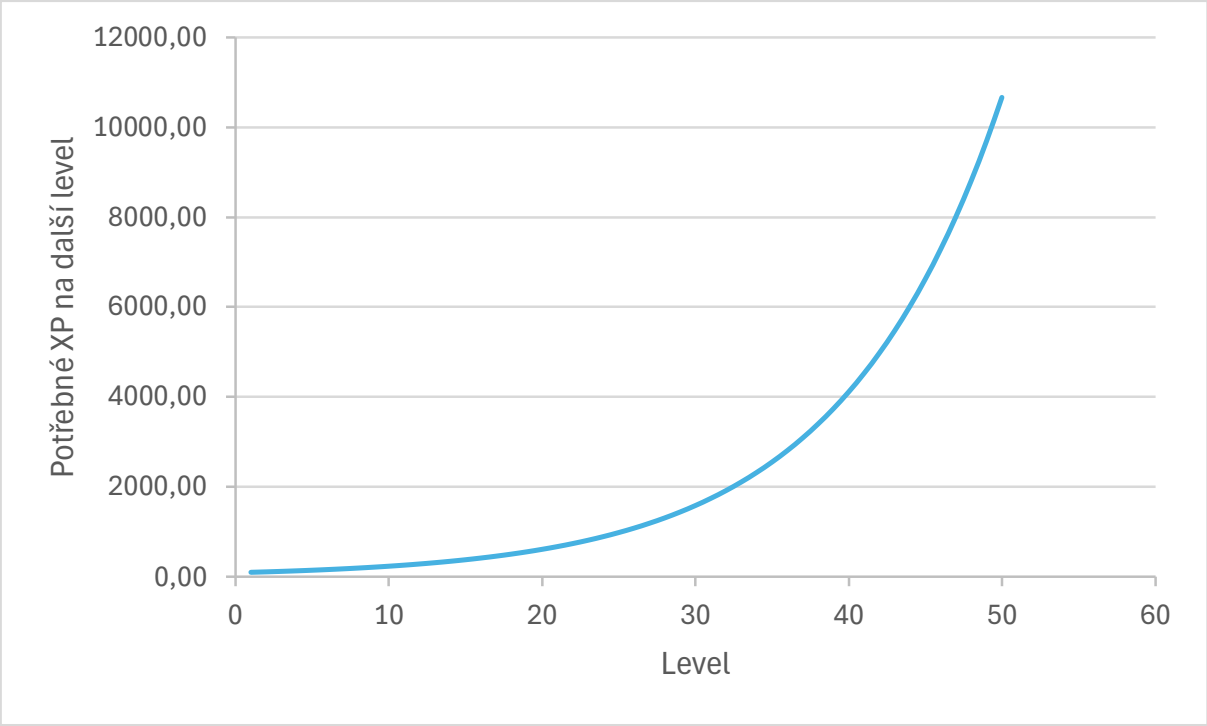
3.6 Gamifikace a výkonová optimalizace

Z hlediska databázového výkonu je výpočet statistik náročný. V projektu byla proto uplatněna strategie denormalizace. V modelu *User.cs* jsou uloženy fyzické sloupce *TotalTasksCompleted* a *TotalTasksCreated*. Tyto čítače jsou inkrementovány pouze jednou – v momentě schválení úkolu. Server pak při zobrazení profilu nebo žebříčku pouze přečte jednu číselnou hodnotu, což je o několik řádů rychlejší než agregační dotazy *COUNT(*)*.

Významnou roli v architektuře hraje služba ILevelingService, která centralizuje matematický model gamifikace. Je registrována s životním cyklem Scoped a je injektována do všech komponent, které vyžadují modifikaci nebo výpočet úrovní uživatele (typicky UserService a AchievementService). Tato centralizace zajišťuje, že výpočet zkušenostních bodů a postupu na vyšší úrovně probíhá pomocí identického algoritmu napříč celou aplikací, čímž je eliminována redundance kódu.

Samotný leveling systém využívá exponenciální matematický vzorec $100 \times 1{,}1^{(Level-1)}$. Pro lepší vizuální představu o dynamice tohoto matematického modelu byl vygenerován graf (viz Obr. 3.6-1: Křivka exponenciálního růstu XP). Z grafického znázornění je jasně patrné, že zatímco v počátečních fázích hry (nižší úrovně) je postup relativně rychlý a slouží k rychlému „zaháčkování“ (onboarding) nového uživatele, u vyšších úrovní se křivka strmě zvedá. K dosažení elitních úrovní je tak vyžadována dlouhodobá a konzistentní aktivita v rámci

komunity. Tento systém efektivně brání příliš rychlému vyčerpání herního obsahu a udržuje dlouhodobou retenci uživatelů. Detailní propočty náročnosti postupu a počet úkolů nutných pro dosažení milníků jsou uvedeny v následujícím přehledu (viz Tab. 3.6-1: Progrese úrovní). Pro zajištění pravidelného příjmu zkušeností byly definovány standardizované zdroje XP bodů, které tvoří základní motor uživatelského rozvoje (viz Tab. 3.6-2: Zdroje zkušeností).



Obr. 3.6-1: Křivka exponenciálního růstu XP

Tab. 3.6-1: Progrese úrovní

Level	XP na další level	Celkem XP	Počet úkolu	Poznámka
1	100	0	2	Start
2	110	100	±2	Velmi rychlý postup
5	146	464	±3	První milník (achievment)
10	236	1358	±5	Střední náročnost
15	380	2810	±8	Vyšší náročnost
20	612	5116	±12	Pokročilý uživatel

Tab. 3.6-2: Zdroje zkušeností

Akce v systému	Hodnota (XP)	Četnost	Význam
Splnění úkolu	50	Opakovaně	Základní motor progrese.
Schválení řešení	0	Opakovaně	Autor získá reputaci, nikoliv XP.
Získání achievementu	50–10000	Jednorázově	Bonus za překonání milníku.

Technická realizace odměňování za specifické úspěchy je zajištěna službou AchievementService. Po každé kritické akci systém vyvolá kontrolu podmínek (princip detekce události pomocí Event-driven logiky) pro 33 unikátních odznaků (viz Tab. 3.6-4: Kompletní seznam achievementů). Tyto odznaky jsou odstupňovány podle vzácnosti (Rarity), která přímo koreluje s výší XP odměny (viz Tab. 3.6-3: Logika odměn za vzácnost). Specifickou logiku vyžadují tzv. Secret Achievements, které sledují časové rámce akcí a podporují v uživateli prvek objevování. Celý proces je úzce integrován do workflow schvalování úkolů, kde po zápisu dat do databáze dochází k automatickému řetězci: přepočet XP → kontrola achievementů → odeslání gratulační notifikace.

Tab. 3.6-3: Logika odměn za vzácnost

Vzácnost	Rozsah odměny (XP)	Charakteristika
Common	50–300	Snadno dosažitelné cíle pro motivaci začátečníků.
Rare	500–1500	Vyžaduje soustavnou aktivitu po dobu týdnů.
Epic	1000–4000	Milníky pro věrné a velmi aktivní uživatele.
Legendary	2500–10000	Maximální mety vyžadující měsíce až roky práce.

Tab. 3.6-4: Kompletní seznam achievementů

Kategorie	Název	Podmínka	Odměna (XP)	Vzácnost
Počet splněných úkolů	Dobrý skutek	1	50	Common
	Pomocná ruka	20	200	Common
	Spasitel	50	500	Rare
	Lokální hrdina	100	1000	Epic
	Státní legenda	250	2500	Legendary
Rychlost splnění	Blesk	1 za 24 h	500	Rare
	Sprinter	5 za 24 h	2000	Epic
	Teleport	20 za 24 h	5000	Legendary
Konzistence *(splnění 1 úkolu za týden po dobu x)*	Vytrvalec	2 týdnů	150	Common
	Nezastavitelný	12 týdnů	1000	Rare
	Perpetuum Mobile	24 týdnů	2500	Epic
Počet vytvořených úkolů	Iniciátor	1	50	Common
	Organizátor	10	250	Common
	Hlas lidu	25	600	Rare
	Starosta	50	1200	Epic
	Architekt komunity	100	3000	Legendary
Milníky úrovní (dosažení úrovně x)	Učeň	2	100	Common
	Pokročilý	5	300	Common
	Expert	10	700	Rare
	Mistr	25	2000	Epic
	Velmistr	50	5000	Legendary
Milníky reputace	Slušný občan	100	100	Common

(dosažení reputace x)	Známá tvář	500	500	Common
	Vlivná osoba	2000	2000	Rare
	Celebrita	5000	5000	Epic
	Bůh Taskify	10000	10000	Legendary
Tajné	Ranní ptáče	Splň úkol mezi 04:00 a 07:00.	250	Rare
	Noční hrdina	Proveď akci mezi 00:00 a 03:00.	250	Rare
	První krok	Přidej si profilovou fotku.	100	Common
	Paparazzi	Přidej 10 fotek k jednomu úkolu.	500	Rare
	Na poslední chvíli	Splň úkol v den deadlinu.	300	Common

Kromě zkušenostních bodů systém pracuje s parametrem reputace, který slouží jako ukazatel důvěryhodnosti v komunitě. Na rozdíl od XP bodů, které uživatel získává za svou aktivitu, reputace reflektuje kvalitu jeho přínosu a ostražitost při hlášení závad. Tento systém je klíčový pro moderaci obsahu, neboť uživatelé s vysokou reputací jsou vnímáni jako pilíře komunity, zatímco sankce od moderátorů vedou k jejímu citelnému snížení. Logika připisování a odebírání reputačních bodů je popsána v níže uvedené tabulce (viz Tab.3.6-5: Systém reputace).

Tab. 3.6-5: Systém reputace

Akce v systému	Hodnota	Cílená osoba	Důvod a význam
Splnění úkolu	+10	Dobrovolník	Odměna za poctivě odvedenou práci.
Vytvoření úkolu	+10	Autor úkolu	Odměna za tvorbu pro ostatní.
Nahlášení (Report)	+15	Oznamovatel	Bonus za ochranu komunity.
Úprava adminem	-20	Autor úkolu	„Žlutá karta“ za chyby v obsahu úkolu.
Smazání adminem	-50	Autor úkolu	„Červená karta“ za hrubé porušení pravidel.
Blokace účtu	Reset	Uživatel	Úplná ztráta důvěryhodnosti.

Pro potřeby analytického dashboardu s obsahem statistik byla implementována backendová logika, která provádí agregaci dat nad tabulkami úkolů a uživatelů. Využitím dotazovacího jazyka LINQ a metod jako *.GroupBy()* nebo *.Sum()* jsou data na straně serveru transformována do odlehčených objektů typu DTO (Data Transfer Object). Tyto objekty obsahují statistické souhrny, jako je počet úkolů v jednotlivých krajích nebo distribuce reputačních bodů napříč komunitou. Tento proces přípravy dat minimalizuje objem přenášených informací mezi serverem a klientem, čímž je zajištěno rychlé vykreslování vizuálních grafů i při rostoucím počtu záznamů v systému.

3.7 Administrace a seeding dat

Pro administraci systému byla vytvořena sada nástrojů umožňující moderaci komunity. Klíčovou funkcí je metoda *UnassignTasksFromUser*, která při zablokování uživatelského účtu automaticky uvolní všechny jeho rozpracované úkoly zpět do stavu Open, čímž chrání integrita komunity. Pro účely testování byl vyvinut *DatabaseSeeder* využívající knihovnu Bogus. Klíčovou inovací je simulace GPS souřadnic podle 14 geografických center (krajů ČR). Pro každý úkol je vypočtena poloha pomocí náhodné odchylky v rozmezí ± 0,05 stupně, což zajišťuje realistické rozložení úkolů na mapě bez jejich překrývání v jednom bodě. Vizuální věrohodnost je umocněna integrací Unsplash API pro tematické fotografie.

```
private async Task UnassignTasksFromUser(string userId)
{
    var tasksToRelease = await _context.Tasks
        .Where(t => t.AssignedToId == userId && t.Status ==
Models.Enums.TaskStatus.InProgress)
        .ToListAsync();

    foreach (var task in tasksToRelease)
    {
        task.AssignedToId = null;
        task.Status = Models.Enums.TaskStatus.Open;
    }

    if (tasksToRelease.Any())
    {
        await _context.SaveChangesAsync();
    }
}
```

Ukázka kódu 3.7-1: Uvolnění rozpracovaných úkolů při blokaci uživatele

3.8 Analýza a řešení kritických logických defektů

3.8.1 Popis první anomálie

Během fáze interního beta-testování aplikace s reálnými uživateli byl identifikován kritický logický defekt v modulu pro správu uživatelských úrovní (UserService). Tato kapitola popisuje proces identifikace, technickou analýzu a následnou opravu této chyby.

Při standardní interakci uživatele s profilem (konkrétně u nahrání profilového obrázku) došlo k neočekávanému nárůstu uživatelské úrovně z hodnoty 1 na hodnotu 58 během jedné databázové transakce. Tento stav byl doprovázen masivním generováním notifikací o dosažení nových milníků, což negativně ovlivnilo stabilitu klientské části aplikace a integritu herní ekonomiky.

Detailní revizí zdrojového kódu metody UpdateLevelAsync bylo zjištěno, že řídicí struktura while, zodpovědná za výpočet postupu, vykazovala dvě zásadní logické vady:

- Absence degradace XP fondu: Algoritmus v každém průchodu cyklu kontroloval, zda celkový počet bodů uživatele přesahuje hranici pro další úroveň, avšak nedocházelo k odečítání (konzumaci) těchto bodů po úspěšném zvýšení úrovně.
- Statické vyhodnocení hranice: Parametr definující cenu za postup byl vypočítán pouze jednou před vstupem do cyklu. V případě zisku většího množství zkušeností tak systém nebral v potaz exponenciální nárůst náročnosti u vyšších úrovní.

3.8.2 Implementace nápravného opatření první anomálie

K odstranění chyby bylo přistoupeno k celkové refaktorizaci metody. Byl implementován princip „bodové ceny za úroveň“, kdy je při každém úspěšném průchodu cyklem od uživatelského konta odečtena přesná hodnota XP odpovídající aktuálnímu stupni progrese. Zároveň byla do těla cyklu vložena reevaluace proměnné *pointsNeededForNext*, čímž byla zajištěna matematická korektnost exponenciálního modelu i při skokovém nárůstu bodů.

Sekundární příčinou nestability systému byla redundance (opakování) logiky pro výpočet úrovní na dvou různých místech v architektuře aplikace (UserService a AchievementService). Zatímco v *UserService* byl defekt opraven, v *AchievementService* zůstala původní chybná implementace, která nezahrnovala dedukci zkušenostních bodů.

Tento incident demonstroval důležitost principu DRY (Don't Repeat Yourself). Pro finální verzi aplikace bylo proto implementováno vyčlenění matematického modelu levelingu do samostatné služby *ILevelingService* (detailně popsáno v kapitole 3.6 Gamifikace a výkonová optimalizace), čímž se předešlo budoucím nekonzistencím a logickým rozporům v datech.

4 Implementace klientské vrstvy a uživatelského rozhraní

V této kapitole je detailně rozebrána implementace prezentační vrstvy webové aplikace Taskify. Hlavní důraz je kladen na vizuální interpretaci dat, interakci s uživatelem a zajištění responzivity napříč různými koncovými zařízeními. Zatímco backendová část primárně řeší logiku a perzistenci dat, frontendová architektura byla navržena s cílem maximalizovat uživatelský komfort, přehlednost a v neposlední řadě bezpečnost prostřednictvím kontrolovaného vykreslování citlivých informací na straně klienta.

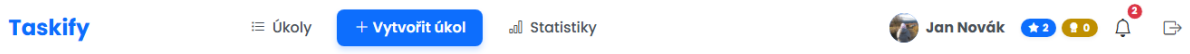
4.1 Architektura prezentační vrstvy a modernizace stylování

Pro vytvoření moderního a intuitivního uživatelského rozhraní (UI) byl jako hlavní stylovací rámec zvolen CSS framework Bootstrap ve verzi 5. Tento nástroj byl vybrán pro svou nativní podporu responzivního designu založeného na přístupu „Mobile-First“. Celé rozložení prvků na obrazovce je primárně řízeno pomocí pokročilého Grid systému a technologie Flexbox. Architektura prezentační vrstvy je centralizována v souboru *_Layout.cshtml*, který slouží jako hlavní šablona definující strukturu hlavičky s navigačním panelem, kontejneru pro samotný obsah (vykreslovaný pomocí direktivy *@RenderBody()* a patičky.

Standardní vizuální podoba frameworku Bootstrap byla ovšem výrazně modifikována, aby odpovídala specifickým estetickým požadavkům komunitní platformy. Klíčovým prvkem této modernizace byla integrace externí typografie prostřednictvím služby Google Fonts. Pro veškeré textové prvky v aplikaci byl nasazen bezpatkový geometrický font „Poppins“. Jeho deklarace byla provedena přímo v hlavičce HTML dokumentu a následně vynucena v globálním souboru *site.css* pomocí pravidla *font-family: 'Poppins', sans-serif !important;*. Tento font se vyznačuje vysokou čitelností a moderním vzhledem, což výrazně přispívá ke snížení kognitivní zátěže uživatele.

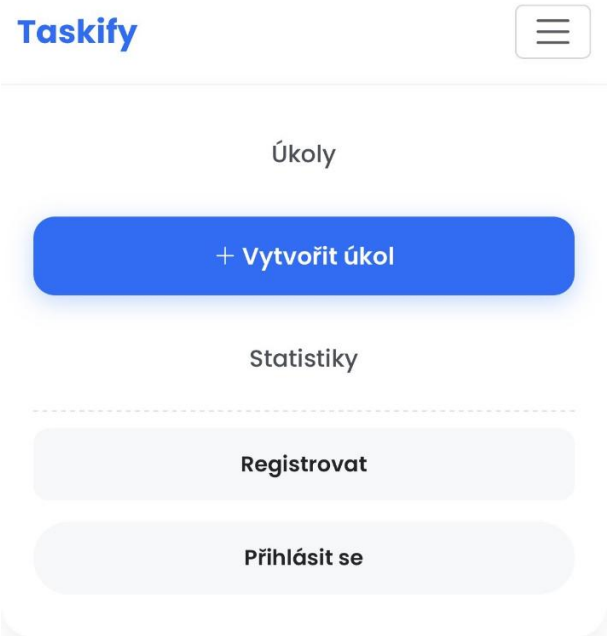
Kromě typografie byl v rámci vlastních kaskádových stylů kladen velký důraz na interaktivitu prvků. Byla přepsána výchozí pravidla pro primární tlačítka (třída *.btn-primary*), kterým byl přiřazen specifický odstín modré barvy (*#0d6efd*), zaoblení rohů (*border-radius: 8px*) a plynulé CSS přechody. Při interakci uživatele (hover efekt) dochází k aplikaci pravidla *transform: translateY(-2px);* v kombinaci s rozšířeným stínem (*box-shadow*), čímž je dosaženo vizuálního efektu "vznášení" tlačítka. Pro eliminaci nechtěných animací u statických prvků byla rovněž definována speciální pomocná třída *.no-anim*.

Vizuální rozdíly v rozhraní a úspěšná implementace responzivity jsou demonstrovány na následujících ukázkách. Na Obr. 4.1-1 je zobrazeno standardní desktopové rozhraní pro přihlášeného uživatele, které zahrnuje personalizované prvky a přístup k administraci. Oproti tomu Obr. 4.1-2 ilustruje zjednodušenou verzi navigačního panelu pro hosty s důrazem na registrační tlačítka. Adaptabilita systému na mobilní zařízení je potvrzena na Obr. 4.1-3 (pohled hosta) a Obr. 4.1-4, který detailně zobrazuje mobilní menu s uživatelským profilem a notifikačním centrem.

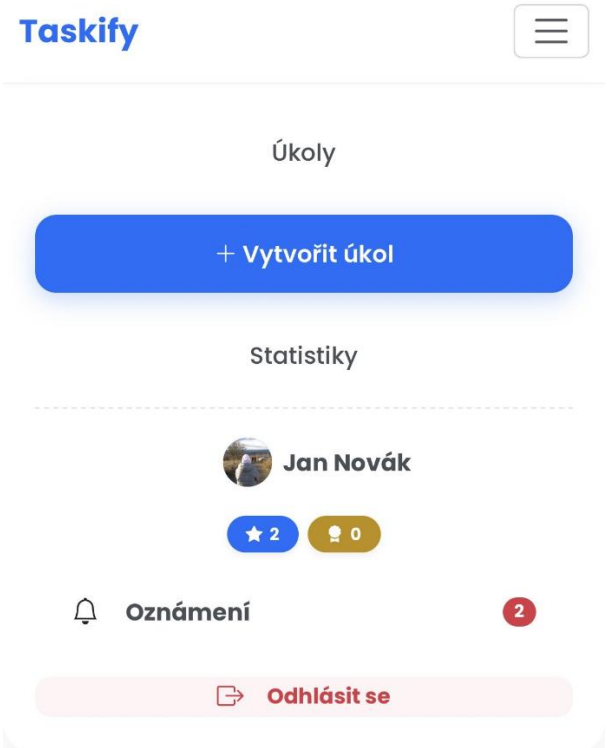


Obr. 4.1-1: Desktopové rozhraní navigace pro přihlášeného uživatele.

Obr. 4.1-2: Mobilní rozhraní navigace



Obr. 4.1-3: Mobilní rozhraní navigace – menu – nepřihlášený uživatel.



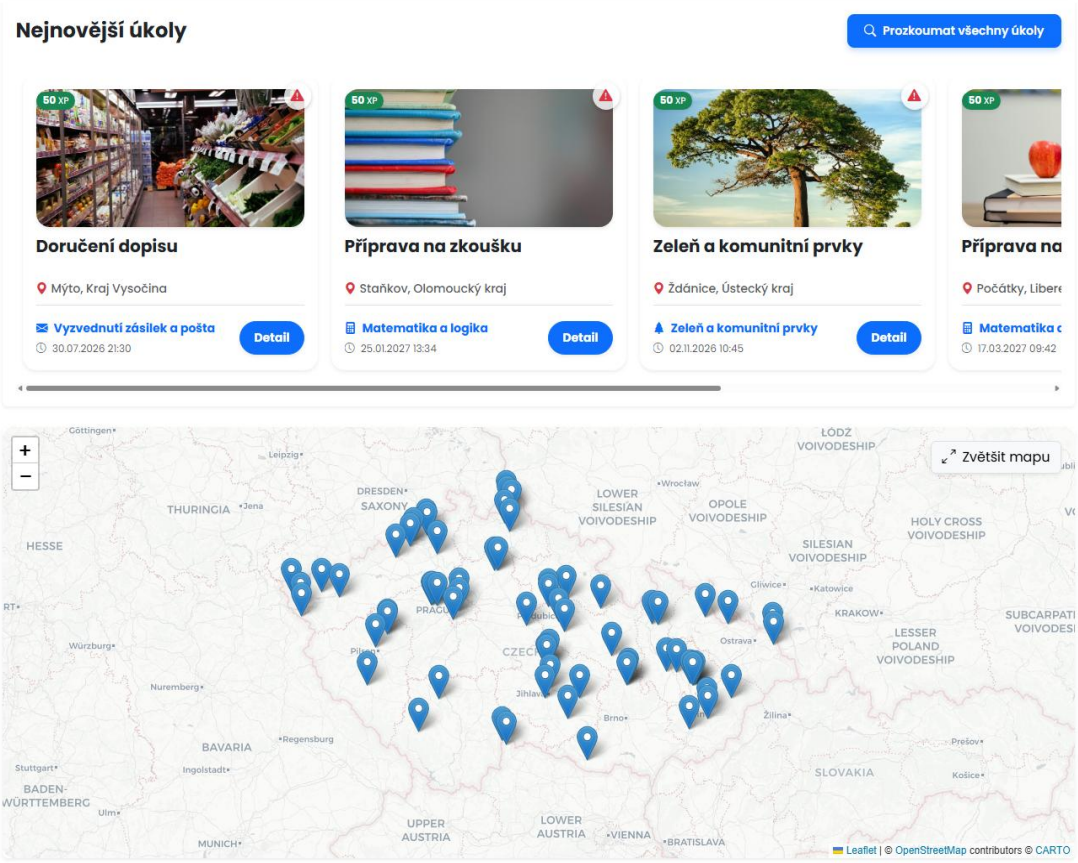
Obr. 4.1-4: Mobilní rozhraní navigace – menu – přihlášený uživatel.

4.2 Integrace mapového rozhraní Leaflet.js

Ústředním prvkem aplikace Taskify, který ji odlišuje od běžných inzertních textových nástěnek, je interaktivní mapové rozhraní. Pro jeho implementaci na straně klienta byla využita odlehčená open-source JavaScriptová knihovna Leaflet.js. Modul mapy je inicializován na domovské stránce (*Index.cshtml*), kde je připraven dedikovaný HTML element * *. Samotná inicializace mapy probíhá asynchronně po načtení dokumentu (DOM) v souboru *dashboard.js*, kde je instancován objekt *L.map* s výchozími souřadnicemi a úrovní přiblížení. Jako primární zdroj mapových dlaždic (Tile Layers) byla zvolena veřejně dostupná služba OpenStreetMap.

Nejkritičtějším technologickým aspektem byla integrace datového toku mezi backendem (C#) a frontendem (JavaScript). Přenos dat je realizován technikou serializace objektů přímo v Razor View. Data o dostupných úkolech jsou transformována pomocí metody *Json.Serialize()* do formátu JSON a vložena jako hodnota datového atributu *data-tasks* přímo do elementu mapy. Klientský skript (*dashboard.js*) následně tento atribut přečte a pomocí metody *JSON.parse()* jej převede zpět na pole JavaScriptových objektů.

Skript následně iteruje nad touto datovou strukturou a pro každý záznam vytváří značku na mapě (*L.marker*). Součástí implementace je i dynamické generování HTML obsahu pro modální vyskakovací okna (tzv. Popup), která se zobrazí po kliknutí na značku. Pomocí JavaScriptu je do šablony (template) vložen název úkolu, kategorie, odměna v XP a profilový obrázek. Výsledný HTML řetězec je následně navázán na instanci markeru metodou *.bindPopup()*. Vizuální výstup domovské stránky kombinující výpis nejnovějších karet úkolů a mapové zobrazení s distribuovanými markery je znázorněn na Obr. 4.2-1.



Obr. 4.2-1: Hlavní rozhraní s mapou Leaflet.js a kartami úkolů.

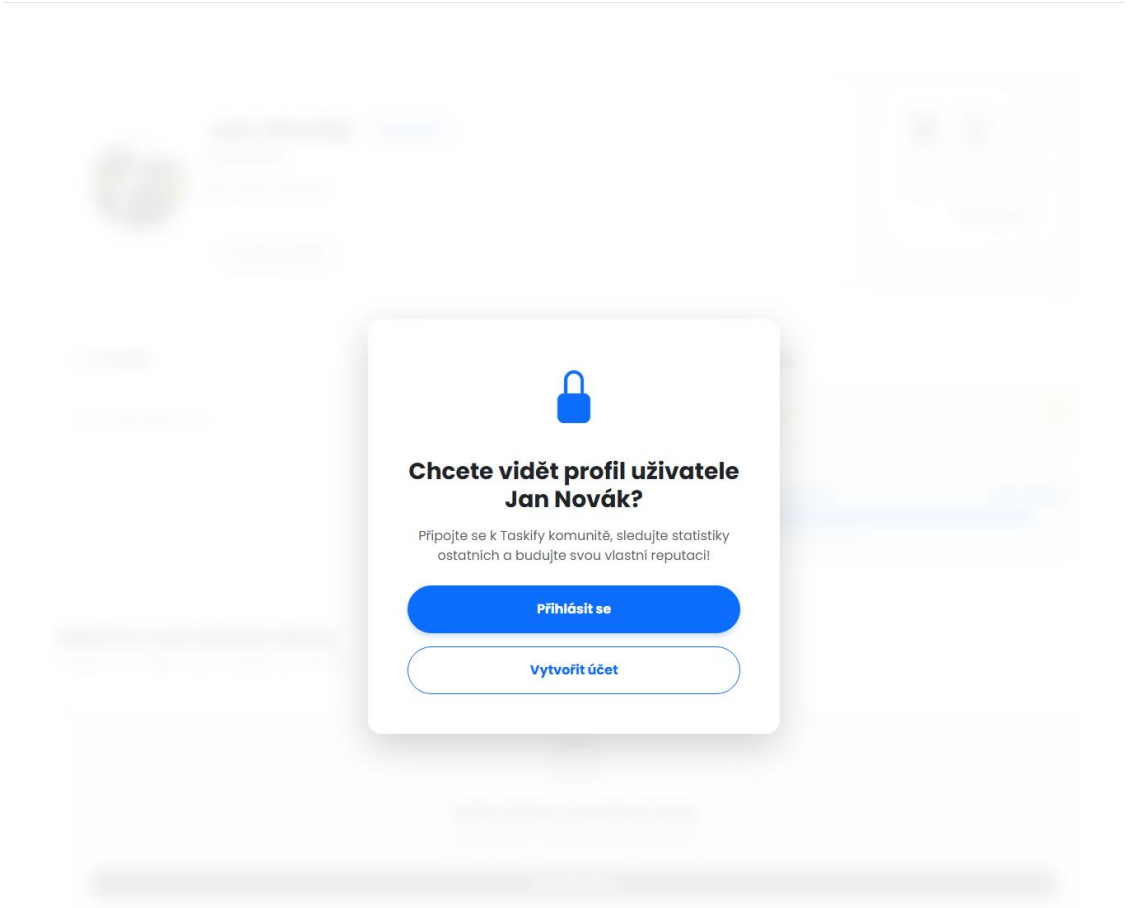
4.3 Server-Side Data Masking a ochrana lokací pro hosty

Vzhledem k povaze komunitní platformy, kde uživatelé sdílejí polohu svého bydliště či místa výkonu úkolu, byla bezpečnost a ochrana soukromí absolutní prioritou. Pro nepřihlášené návštěvníky (role Guest) byla implementována robustní ochranná vrstva označovaná jako „Server-Side Data Masking“ (maskování dat na straně serveru). K záměrné obfuskaci (zamlžení) dat dochází výhradně na serveru pomocí vložených C# výrazů v souboru *Index.cshtml*, ještě před samotnou serializací dat do JSON formátu.

Pokud stavový objekt *User.Identity.IsAuthenticated* vrací hodnotu *false*, je spuštěn algoritmus, který aplikuje matematický šum na přesné GPS souřadnice úkolu. Konkrétně je k původní zeměpisné šířce (*Latitude*) a délce (*Longitude*) přičtena náhodně vygenerovaná hodnota (*new Random().NextDouble() * 0.002*). Tímto způsobem je bod na mapě posunut v rámci rádiusu o velikosti několika set metrů. Dále dochází ke skrytí názvu kategorie (nahrazeno řetězcem "???") a vynulování zobrazené XP odměny. Útočník tak nemá žádnou možnost extrahovat přesná data ze zdrojového kódu stránky, jelikož prohlížeč obdržel pouze již znehodnocené souřadnice.

Tento bezpečnostní mechanismus je úzce spjat s designem uživatelského prožitku (UX) prostřednictvím UI komponenty zvané „Login Gate“. Pokud se neautentizovaný uživatel pokusí otevřít detail chráněného uživatelského profilu nebo mapy, obsah stránky je vizuálně znehodnocen pomocí CSS třídy *.blur-content*, která aplikuje pravidlo *filter: blur(6px)*. Následně je přes obrazovku překryt element *.login-gate-overlay* s vysokou hodnotou *z-index*. Toto modální okno s animací *slideUp* (posun zespodu) informuje uživatele o nutnosti ověření identity a slouží jako primární konverzní trychtýř platformy, neboť obsahuje prominentní výzvy k akci (Call to Action tlačítka) pro registraci a přihlášení.

Efekt bezpečnostního mechanismu „Login Gate“ v kombinaci s rozostřením citlivého obsahu je vizualizován na Obr. 4.3-1. Tento přístup nejen chrání soukromí komunity, ale zároveň slouží jako motivační prvek pro registraci nových uživatelů, kterým je obsah zpřístupněn až po úspěšné autentizaci.



Obr. 4.3-1: Mechanismus Login Gate s efektem rozostření obsahu.

4.4 Analytika a vizualizace dat prostřednictvím Chart.js

Pro sekci uživatelských statistik byla do projektu integrována JavaScriptová knihovna Chart.js, která zajišťuje hardwarově akcelerované vykreslování vizualizací nad HTML5 Canvasem. Inicializace grafů probíhá v souboru *Statistics/index.js*. Pro vizuální odlišení grafů od zbytku aplikace byl v konfiguraci knihovny globálně nastaven font „Plus Jakarta Sans“ (*Chart.defaults.font.family = "'Plus Jakarta Sans', sans-serif";*).

Vzhledem k obrovskému množství dat generovaných systémem je vizualizační vrstva silně závislá na komplexní datové agregaci na backendu. Prostřednictvím pokročilých LINQ dotazů v jazyce C# (využívajících metody *.GroupBy()*, *.Select()* a paměťovou materializaci dat) jsou surová data o úkolech transformována do struktury DTO (Data Transfer Object). Tento objekt obsahuje pouze hotová pole hodnot a štítků, která jsou na frontendu zpřístupněna přes globální proměnnou *window.statsData*.

Implementovány byly čtyři základní typy vizualizací:

1. **Trendový graf (Line Chart):** Vykresluje nárůst počtu nových úkolů v čase. Pro vizuální zjemnění dat je aplikován parametr *tension: 0.4*, který zajišťuje plynulé proložení křivky mezi datovými body, a plocha pod křivkou je vyplněna barvou s nízkou opacitou (*fill: true*).
2. **Stavový graf (Doughnut Chart):** Tento prstencový graf slouží k zobrazení proporčního zastoupení stavů úkolů. Využívá vlastnost *cutout: '75%'* k vytvoření tenkého prstence, který působí moderněji než klasický koláčový graf.
3. **Hodinová distribuce (Bar Chart):** Sloupcový graf s fixním polem dvaceti čtyř štítků (0:00 až 23:00) vizualizuje četnost odevzdávání úkolů v závislosti na denní době.
4. **Týdenní aktivita (Bar Chart):** Ukazuje absolutní počet interakcí v jednotlivých dnech v týdnu pro identifikaci nejvíce exponovaných dnů pro provoz serveru.

Výstup analytického modulu je zachycen na Obr. 4.4-1. Zobrazené grafy využívají knihovnu Chart.js k interpretaci agregovaných dat o aktivitě uživatelů a distribuci úkolů v čase, což umožňuje efektivní monitoring zdraví celé platformy

Obr. 4.4-1: Dashboard s grafy Chart.js.

4.5 Vizuální prezentace gamifikačních mechanismů v UI

Zatímco výpočetní logika postupu na další úroveň (leveling) a udělování achievementů je striktně řízena backendovými službami (jak bylo popsáno v předchozí kapitole), frontend je zodpovědný za transformaci těchto dat do poutavé vizuální zpětné vazby. Architektura uživatelského profilu (*UserProfile.cshtml*) byla navržena tak, aby okamžitě komunikovala prestiž a aktivitu daného uživatele v komunitě.

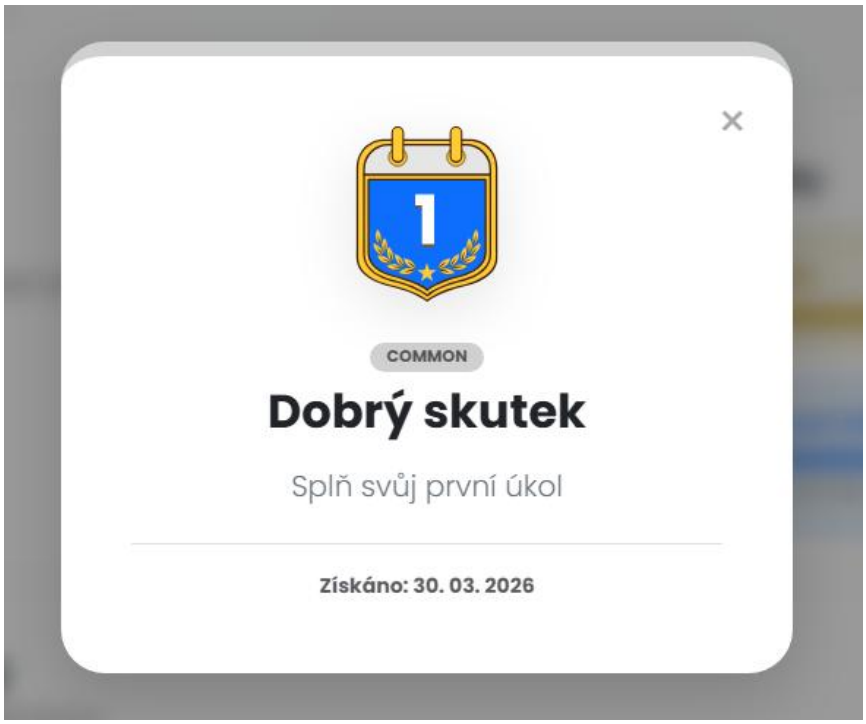
Základním stavebním kamenem vizualizace herního postupu je komponenta Progress Baru. Tento ukazatel je definován v HTML pomocí struktury využívající třídy *progress* a *progress-bar*. Vlastní šířka (naplněnost) ukazatele je dynamicky počítána na serveru z poměru aktuálních bodů (*Points*) a bodů potřebných pro další úroveň (*NextLevelPoints*). Vypočtená procentuální hodnota je injektována přímo do inline atributu *style="width: @xpPercent%;"*. Pro zobrazení reputace uživatele byl implementován mechanismus, který při negativním skóre dynamicky mění CSS třídy rámečku z neutrální *bg-custom-warning-light* na výstražnou *bg-danger-subtle* a doplňuje profil varovnou ikonografií upozorňující komunitu na nedůvěryhodného člena.

Vizuální design systému odznaků (Achievementů) je postaven na mřížkovém rozložení (*.achievement-grid*). Nejdůležitějším prvkem je klasifikace vzácnosti (Rarity). Model každého achievementu obsahuje enumerátor (např. Common, Rare, Epic, Legendary), který se v Razor View překládá do specifické CSS třídy (např. *rarity-epic*). Tato CSS třída následně v souboru definuje barevné schéma ohraničení, záři modálního okna (box-shadow) a barvu typografie.

Pro interakci s achievementy byla v JavaScriptu naprogramována funkce *showAchievementDetail()*, která manipuluje se skrytým modálním oknem *#achievement-overlay*. Po kliknutí na ikonu odznaku se dynamicky změní obsah elementů *<img>* a ** v overlayi podle parametrů předaných funkcí, čímž je dosaženo rychlého vykreslení detailu bez nutnosti obnovování stránky.

Vizuální zpracování ikonografie achievementů bylo pojato s důrazem na funkční demonstraci systému v rámci aplikace. Přibližně deset klíčových grafických návrhů odznaků bylo vytvořeno externím spolupracovníkem, zatímco zbývající část ikon tvoří dočasné zástupné symboly (placeholdery). Vzhledem k technickému zaměření maturitní práce nebylo autorské vytvoření kompletní sady grafiky prioritou; hlavním cílem bylo korektní propojení backendové logiky s prezentační vrstvou a vizuální ověření, že systém odměn je v uživatelském rozhraní plně integrovaný a přehledný.

Vizuální zpětná vazba při interakci se systémem gamifikace je znázorněna na Obr. 4.5-1. Modální okno zobrazuje detail odemčeného ocenění s využitím specifického stylování podle vzácnosti (Rarity) a doprovodnými informacemi o datu získání, což posiluje motivační aspekt aplikace.



Obr. 4.5-1: Detailní zobrazení odemčeného achievementu.

4.6 Pokročilá logika interaktivních formulářů

Vytváření úkolů v aplikaci Taskify představuje jednu z nejkomplexnějších částí klientské vrstvy. Soubor *create.js* řídí celou řadu procesů, které zajišťují integritu dat a vysoký uživatelský standard ještě před odesláním požadavku na server.

4.6.1 Správa multimédií a klientská validace

Aplikace umožňuje nahrávání více fotografií k jednomu úkolu. Implementovaná JavaScriptová logika v reálném čase monitoruje změny v elementu *#imageUploads*. Při výběru souborů dochází k okamžité klientské validaci typu souboru; specificky je blokován formát GIF, aby byla zachována serióznost vizuálního obsahu. Po úspěšné validaci jsou soubory zpracovány pomocí objektu *FileReader*, který generuje náhledy v podobě base64 řetězců. Tyto náhledy jsou dynamicky injektovány do elementu *#imagePreview*, přičemž první nahraný obrázek je automaticky vizuálně označen štítkem „Hlavní“, což uživateli indikuje, která fotografie bude použita v náhledech karet úkolů.

4.6.2 Integrace externích UI komponent

Pro zajištění validních časových údajů byl do formuláře integrován *AirDatepicker*. Tato komponenta nahrazuje standardní prohlížečové vstupy pro datum a čas, přičemž je plně lokalizována do českého jazyka (*datepickerLocaleCs*). Logika na frontendu automaticky nastavuje minimální povolené datum na aktuální den a předvyplňuje termín na sedm dní dopředu, což snižuje počet interakcí nutných k odeslání formuláře.

4.7 Geokódování a interakce s Nominatim API

Mapa Leaflet.js ve formuláři pro vytvoření úkolu neslouží pouze k zobrazení, ale jako aktivní nástroj pro sběr lokalizačních dat. Implementace využívá dva klíčové procesy:

1. **Reverzní geokódování:** Po kliknutí uživatele do libovolného místa na mapě je zachycena událost *map.on('click')*. JavaScript následně odešle asynchronní požadavek na API služby Nominatim (OpenStreetMap). Vrácená JSON data obsahující město, ulici, číslo popisné a PSČ jsou automaticky rozparsována a vepsána do skrytých polí formuláře. Tímto je eliminována nutnost manuálního vypisování adresy uživatelem.
2. **Přímé geokódování:** V opačném směru může uživatel zadat textovou adresu do vyhledávacího pole. Skript po stisku tlačítka vyhledávání kontaktuje API, získá souřadnice a následně přesune kameru mapy (*map.setView*) a umístí marker na nalezenou lokaci.

Tato obousměrná vazba mezi mapou a textovými poli zajišťuje, že každý úkol v databázi disponuje přesnými geografickými souřadnicemi nezbytnými pro globální mapu systému.

4.8 Asynchronní systém hlášení a správa incidentů

Z bezpečnostních důvodů je v aplikaci implementován systém pro nahlašování nevhodného obsahu (Reporting). Tato funkcionalita je na frontendu realizována prostřednictvím souboru *report_modal.js*. Po kliknutí na ikonu výstražného trojúhelníku na kartě úkolu je vyvolána funkce *openReportModal(taskId)*, která dynamicky naplní skryté pole v univerzálním modálním okně ID daného úkolu. Vizuální podoba tohoto rozhraní, které uživateli nabízí výběr kategorií porušení pravidel (např. Nevhodný obsah, Spam či Podvod), je zachycena na Obr. 4.8-1.

Obr. 4.8-1: Modální okno pro hlášení obsahu..

Odeslání hlášení probíhá asynchronně prostřednictvím rozhraní Fetch API, které umožňuje komunikaci s endpointem /Reports/Report bez nutnosti znovunačtení stránky. Skript sbírá data o důvodu hlášení a doplňujícím popisu, přičemž pro bezpečnost proti útokům typu Cross-Site Request Forgery (CSRF) extrahuje z DOMu verifikační token (*__RequestVerificationToken*). Po úspěšném zpracování požadavku na serveru skript vyvolá globální funkci *showStatusAlert()*, která uživateli zobrazí dočasné oznámení o výsledku akce, aniž by došlo k přerušení jeho práce na aktuální stránce.

```
const response = await fetch('/Reports/Report', {
    method: 'POST',
    headers: {
        'Content-Type': 'application/json',
        'RequestVerificationToken': token
    },
    body: JSON.stringify(data)
});
```

Ukázka kódu 4.8-1: Asynchronní odeslání hlášení s JSON serializací a CSRF ochranou.

4.9 Modularita a design systému „Task Card“

Vizuální reprezentace úkolu v seznamech je řešena pomocí znovupoužitelné komponenty `_TaskCardPartial.cshtml`. Tento modulární prvek je navržen tak, aby mohl existovat ve dvou stavech: standardním a mini (využívaném v dashboardu nebo na profilu).

Design karty využívá moderní CSS techniky jako *aspect-ratio: 16/9* a *object-fit: cover* pro zachování jednotného vizuálního řádu bez ohledu na původní rozměry nahraných fotografií. Interaktivita je v hlavní galerii úkolů (*/Tasks/Index*) rozšířena o dynamické procházení fotografií (Image Cycling) realizované pomocí JavaScriptu.

Klíčovým prvkem je využití datového atributu *data-images*, do kterého jsou v Razor View serializovány URL adresy všech nahraných snímků. Jak ukazuje Ukázka kódu 4.9-1, skript po najetí myši (mouseenter) aktivuje interval, který v pravidelných cyklech mění zdrojový obraz (src) s využitím plynulého přechodu průhlednosti (opacity).

Součástí implementace je také preloading (přednačtení) všech snímků do mezipaměti prohlížeče při inicializaci stránky, což eliminuje problikávání bílých ploch při změně zdroje. Tato interaktivita je záměrně omezena pouze na hlavní seznam úkolů, aby nedocházelo k nadbytečné zátěži DOM prvků v dashboardu s mapovým rozhraním.

```
cardElement.addEventListener('mouseenter', () => {
    interval = setInterval(() => {
        currentIndex = (currentIndex + 1) % images.length;
        img.style.opacity = '0.3';
        setTimeout(() => {
            img.src = images[currentIndex];
            img.style.opacity = '1';
        }, 200);
    }, 2000);
});
```

Ukázka kódu 4.9-1: Logika dynamické rotace obrázků v komponentě Task Card.

4.10 Dynamické filtrování a vyhledávací bar

Stránka se seznamem úkolů *Tasks/Index.cshtml*) obsahuje sofistikovaný vyhledávací bar, který kombinuje tři typy filtrů:

1. **Full-textové vyhledávání:** Klasický textový vstup pro dotazování v názvech a popisech.
2. **Hierarchický výběr kategorií:** Implementace využívá víceúrovňové rozbalovací menu (dropdown). Pomocí JavaScriptu v souboru *categories-dropdown.js* je zajištěno, že po kliknutí na podkategorii se automaticky aktualizuje skrytý vstup formuláře a vizuální štítek na tlačítku.

3. **Regionální filtr:** Dynamicky generovaný seznam krajů České republiky, který po výběru okamžitě odesílá formulář pro zobrazení lokálně relevantních výsledků.

Vizuální podoba tohoto vyhledávacího baru je definována v *tasks.css*, kde je využito zaoblení s extrémním radiusem (*border-radius: 50rem*) pro dosažení moderního vzhledu připomínajícího vyhledávací pole předních světových platforem.

4.11 Real-time správa oznámení a práce s DOMem

Systém notifikací v aplikaci Taskify je plně integrován do uživatelského rozhraní v souboru *site.js*. Tato část klientské vrstvy řeší několik klíčových interakcí:

- **Aktualizace badge:** Při označení notifikace jako přečtené skript vypočítává novou hodnotu pro číselný indikátor v navigačním panelu. Pokud počet klesne na nulu, indikátor je z DOMu zcela odstraněn pomocí metody *.remove()*.
- **Načítání obsahu (Load More):** Pro optimalizaci výkonu je seznam oznámení načítán po částech. Funkce *loadMoreNotifications* využívá Fetch API k dotazování na částečné HTML (tzv. Partial View), které následně vkládá na konec seznamu metodou *insertAdjacentHTML*. Tento přístup šetří přenesená data a zvyšuje plynulost aplikace.
- **Vizuální indikace stavu:** Přečtené notifikace ztrácejí své barevné zvýraznění (*bg-light-blue*) a modrý bod značící novinku, což probíhá okamžitě na klientské straně pomocí manipulace s CSS třídami.

Závěr

Seznam ilustrací

Obr. 1.4-1: Uživatelský profil 12
Obr. 1.4-2: Sbírka achievementů 12
Obr. 2.3-1: UML Use Case diagram 15
Obr. 3.3-1: ER Diagram databáze Taskify 20
Obr. 3.6-1: Křivka exponenciálního růstu XP 23
Obr. 4.1-1: Desktopové rozhraní navigace pro přihlášeného uživatele. 28
Obr. 4.1-2: Mobilní rozhraní navigace 28
Obr. 4.1-3: Mobilní rozhraní navigace – menu – nepřihlášený uživatel. 29
Obr. 4.1-4: Mobilní rozhraní navigace – menu – přihlášený uživatel. 29
Obr. 4.2-1: Hlavní rozhraní s mapou Leaflet.js a kartami úkolů. 31
Obr. 4.3-1: Mechanismus Login Gate s efektem rozostření obsahu. 32
Obr. 4.4-1: Dashboard s grafy Chart.js. 34
Obr. 4.5-1: Detailní zobrazení odemčeného achievementu. 36
Obr. 4.8-1: Modální okno pro hlášení obsahu.. 38

Seznam tabulek

Tab. 3.6-1: Progrese úrovní 23
Tab. 3.6-2: Zdroje zkušeností 23
Tab. 3.6-3: Logika odměn za vzácnost 24
Tab. 3.6-4: Kompletní seznam achievementů 24
Tab. 3.6-5: Systém reputace 25

Seznam použitých zdrojů a literatury